Lab 11 — ตัวนับขึ้น/ลงบน 7-Segment ด้วย RV32I SoC

บอร์ด Altera DE1 (Cyclone II EP2C20F484C7)

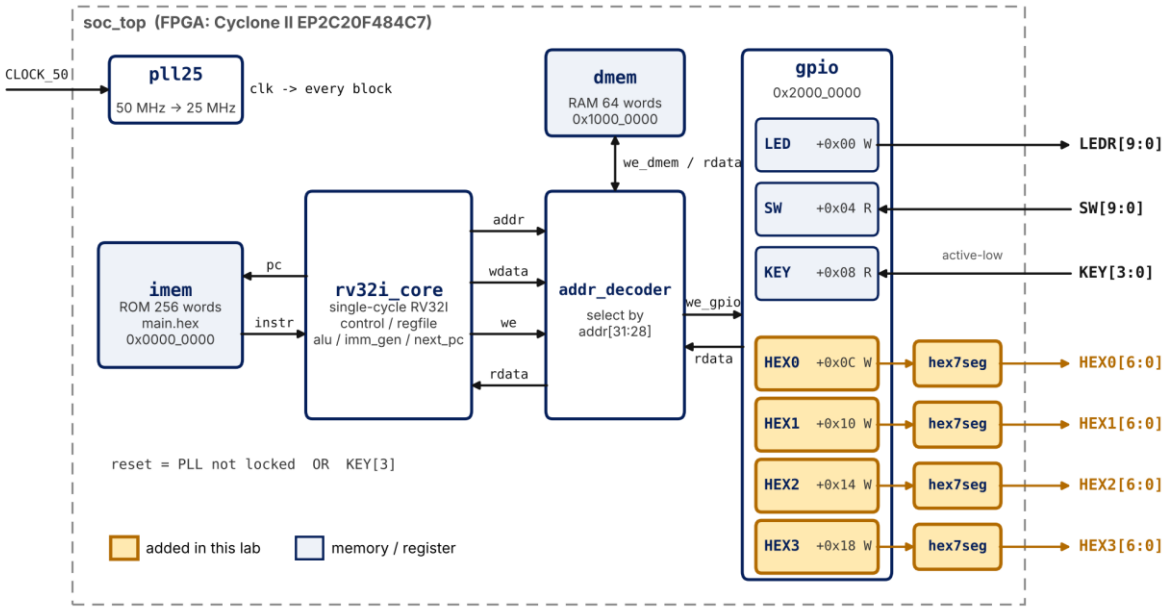
ชื่อ–สกุล	รหัสนิสิต	คะแนน

1. วัตถุประสงค์

- เพิ่ม peripheral ตัวใหม่ (7-segment) เข้า SoC และกำหนด address ให้ register
- เขียนโปรแกรม C แบบ bare-metal อ่านปุ่มกดและแสดงผลผ่าน memory-mapped I/O
- Build โปรแกรม C เป็น `main.hex` ตรวจด้วย simulation และทดสอบบนบอร์ด DE1

2. โครงสร้างระบบ

ระบบตั้งต้นคือโปรเจกต์ **`de1_rv32i_soc`** (โปรแกรมไฟวิ่ง) ซึ่งมี LED, SW และ KEY อยู่แล้ว
ในแล็ปนี้ให้เพิ่มส่วนที่ระบายสีเหลืองในรูป คือ register ของ HEX0–HEX3 ซึ่งแต่ละหลักมี address ของตัวเอง และตัวแปลง **`hex7seg`** หลักละ 1 ตัว



Module	ไฟล์	Address	หน้าที่
`pll25`	`pll25.v`	–	สร้าง clock 25 MHz จาก CLOCK_50

Module	ไฟล์	Address	หน้าที่
`rv32i_core`	`rv32i_core.v`	–	CPU RV32I แบบ single-cycle จาก Chapter 10
`imem`	`imem.v`	`0x0000_0000`	ROM เก็บโปรแกรม โหลดจาก `main.hex`
`dmem`	`dmem.v`	`0x1000_0000`	RAM สำหรับตัวแปรและ stack
`addr_decoder`	`addr_decoder.v`	–	เลือกอุปกรณ์จาก `addr[31:28]` และเลือก `rdata`
`gpio`	`gpio.v`	`0x2000_0000`	register ของ LED, SW, KEY และ HEX0–HEX3
`hex7seg`	`hex7seg.v`	–	แปลงเลข 4 บิตเป็น 7 segment (active-low)

Memory map ของ GPIO

Address	Offset	`addr[4:2]`	Register	ทิศทาง	ความหมาย
`0x2000_0000`	`+0x00`	`3'b000`	LED	เขียน	LEDR[9:0]
`0x2000_0004`	`+0x04`	`3'b001`	SW	อ่าน	SW[9:0]
`0x2000_0008`	`+0x08`	`3'b010`	KEY	อ่าน	KEY[3:0] โดย 1 = กด
`0x2000_000C`	`+0x0C`	`3'b011`	HEX0	เขียน	เลข 4 บิตของ HEX0
`0x2000_0010`	`+0x10`	`3'b100`	HEX1	เขียน	เลข 4 บิตของ HEX1
`0x2000_0014`	`+0x14`	`3'b101`	HEX2	เขียน	เลข 4 บิตของ HEX2
`0x2000_0018`	`+0x18`	`3'b110`	HEX3	เขียน	เลข 4 บิตของ HEX3

register ของ HEX แต่ละตัวเก็บเลข 4 บิต (`wdata[3:0]`) ของหลักนั้น แล้ว `hex7seg` แปลงเป็น 7 segment เดิม GPIO มี 3 register จึงใช้ `addr[3:2]` เลือก เมื่อเพิ่มเป็น 7 register ต้องขยายเป็น `addr[4:2]`

3. โจทย์

เปลี่ยนโปรแกรมไฟวิ่งให้เป็นตัวนับ 1 หลัก (0–9) แสดงผลบน HEX0 โดยมีการทำงานดังนี้

ปุ่ม	การทำงาน
KEY0	กด 1 ครั้ง นับขึ้น 1 (ถึง 9 แล้ววนกลับ 0)
KEY1	กด 1 ครั้ง นับลง 1 ต่อจากค่าปัจจุบัน (ถึง 0 แล้ววนกลับ 9)

ปุ่ม	การทำงาน
KEY3	reset ค่ากลับเป็น 0

ข้อกำหนด

1. กดค้างนานเท่าใดก็นับเพียง 1 ครั้ง
2. HEX3–HEX1 แสดง 0 และ LEDR แสดงค่าเดียวกันเป็นเลขฐานสอง
3. ตัวแปลงเลขเป็น segment ทำด้วย hardware (**hex7seg.v**) โปรแกรม C เขียนเฉพาะค่าตัวเลขลง register ของแต่ละหลัก เช่น **HEX0 = count;**
4. โปรแกรม C ใช้ **unsigned int** เท่านั้น เปรียบเทียบด้วย == และ != เท่านั้น ไม่ใช้ * / % ไม่ใช้ตาราง **const** และตัวแปร global ที่มีค่าเริ่มต้น

ตัวอย่าง: กด KEY0 สามครั้งได้ 3 ⟶ กด KEY1 หนึ่งครั้งได้ 2 ⟶ กด KEY1 อีกสามครั้งได้ 1, 0, 9

4. สิ่งที่ต้องทำ

ไฟล์	สิ่งที่ต้องทำ
rtl/hex7seg.v	คัดลอกจากไฟล์ประกอบแล็ป (ไม่ต้องแก้)
rtl/gpio.v	ขยาย **addr** เป็น **[4:2]**, เพิ่มพอร์ต **hex0**–**hex3**, register 4 บิต 4 ตัวที่ offset **0x0C 0x10 0x14 0x18** และ instance **hex7seg** 4 ตัว
rtl/soc_top.v	เพิ่มพอร์ต **HEX0**–**HEX3** ต่อเข้า **gpio_u**, เปลี่ยน **.addr(addr[3:2])** เป็น **addr[4:2]** และย้าย reset จาก KEY1 ไป KEY3
quartus/de1_soc.qsf	เพิ่มไฟล์ **hex7seg.v** และ pin ของ HEX0–HEX3 ตามตารางด้านล่าง
sw/gpio.h	เพิ่ม **HEX0 HEX1 HEX2 HEX3** ที่ **GPIO_BASE + 0x0C 0x10 0x14 0x18**
sw/main.c	เขียนโปรแกรมตัวนับตามโจทย์
tb/tb_soc.v	แทนด้วยไฟล์ประกอบแล็ป
scripts/build.bat	ลบ 2 บรรทัดที่มีคำว่า **main_sim**
scripts/sim.bat	เพิ่ม **../rtl/hex7seg.v** ในบรรทัด **set RTL=...**

Pin ของ 7-segment (บิต 0 = segment a ถึง บิต 6 = segment g) ให้ตรวจกับ DE1 User Manual ก่อนใช้

	[0] a	[1] b	[2] c	[3] d	[4] e	[5] f	[6] g
HEX0	J2	J1	H2	H1	F2	F1	E2
HEX1	E1	H6	H5	H4	G3	D2	D1

	[0] a	[1] b	[2] c	[3] d	[4] e	[5] f	[6] g
HEX2	G5	G6	C2	C1	E3	E4	D3
HEX3	F4	D5	D6	J4	L8	F3	D4

แนวทาง: loop หลักวนเร็วกว่านิ้วคนมาก จึงต้องนับเฉพาะรอบที่ปุ่มเปลี่ยนจากไม่กดเป็นกด โดยเก็บค่า KEY ของรอบก่อนไว้เทียบกับรอบปัจจุบัน

5. Build, Simulation และลงบอร์ด

```
cd de1_rv32i_soc
set CROSS=riscv-none-elf-
scripts\build.bat
scripts\sim.bat
```

1. `build.bat` ต้องแสดง `check_isa: OK` และ `build: done`
2. `sim.bat` ต้องแสดง `tb_core: PASS` และ `tb_soc: PASS`
3. เปิด `quartus\de1_soc.qpf` แล้ว Compile จากนั้นโปรแกรม `output_files\de1_soc.sof` ลงบอร์ด